Київський національний університет імені Тараса Шевченка

Кафедра мережевих і Інтернет технологій

Лабораторна робота №9

з дисципліни

«Технології створення програмних продуктів»

ШАБЛОН CHAIN OF RESPONSIBILITY

Група: МІТ-м11

Студент: Переверзев Олексій

Київ 2026

Завдання: Створіть систему обробки запитів для служби підтримки програмного забезпечення. Система повинна включати кілька рівнів обробників: базовий технічний спеціаліст, середній технічний спеціаліст і старший технічний спеціаліст. Кожен рівень може обробляти певні типи запитів. Якщо запит не може бути оброблений на поточному рівні, він передається до наступного обробника у ланцюжку.

Код реалізації:

```
using System;

namespace ChainOfResponsibilityTask
{
    public class SupportRequest
    {
        public int DifficultyLevel { get; set; }
        public string Description { get; set; }

        public SupportRequest(int difficultyLevel, string description)
        {
            DifficultyLevel = difficultyLevel;
            Description = description;
        }
    }

    abstract class SupportHandler // Handler
    {
        protected SupportHandler successor;

        public void SetSuccessor(SupportHandler successor)
        {
            this.successor = successor;
        }

        public abstract void HandleRequest(SupportRequest request);
    }

    class BasicSupport : SupportHandler // ConcreteHandler
    {
        public override void HandleRequest(SupportRequest request)
        {
            if (request.DifficultyLevel == 1)
            {
                Console.WriteLine($"[Базовий спеціаліст] вирішив проблему:
{request.Description}");
            }
            else if (successor != null)
            {
                Console.WriteLine($"[Базовий спеціаліст] бракує доступу. Передача запиту
далі...");
                successor.HandleRequest(request);
            }
        }
    }

    class MidLevelSupport : SupportHandler
    {
        public override void HandleRequest(SupportRequest request)
        {
            if (request.DifficultyLevel == 2)
            {
                Console.WriteLine($"[Середній спеціаліст] вирішив проблему:
{request.Description}");
            }
```

```
            else if (successor != null)
            {
                Console.WriteLine($"[Середній спеціаліст] бракує компетенції. Передача
запиту далі...");
                successor.HandleRequest(request);
            }
        }
    }

    class SeniorSupport : SupportHandler
    {
        public override void HandleRequest(SupportRequest request)
        {
            if (request.DifficultyLevel >= 3)
            {
                Console.WriteLine($"[Старший спеціаліст] вирішив КРИТИЧНУ проблему:
{request.Description}");
            }
            else if (successor != null)
            {
                successor.HandleRequest(request);
            }
            else
            {
                Console.WriteLine($"[Система] Запит '{request.Description}' неможливо
обробити.");
            }
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Служба підтримки ПЗ (Chain of Responsibility)
===\n");

            SupportHandler basic = new BasicSupport();
            SupportHandler mid = new MidLevelSupport();
            SupportHandler senior = new SeniorSupport();

            basic.SetSuccessor(mid);
            mid.SetSuccessor(senior);

            SupportRequest[] requests = {
                new SupportRequest(1, "Скинути пароль користувача"),
                new SupportRequest(2, "Помилка виконання SQL-запиту в базі даних"),
                new SupportRequest(3, "Повне падіння головного сервера"),
                new SupportRequest(1, "Очистити кеш браузера клієнта")
            };

            foreach (SupportRequest request in requests)
            {
                Console.WriteLine($"\n--- Новий запит: {request.Description} (Рівень:
{request.DifficultyLevel}) ---");
                basic.HandleRequest(request);
            }

            Console.ReadKey();
        }
    }
}
```

У цьому завданні реалізовано поведінковий шаблон Chain of Responsibility. Згідно з вимогами, створено ланцюжок з трьох рівнів підтримки. Всі обробники наслідують абстрактний клас SupportHandler і містять посилання на свого наступника (successor) . Клієнтська програма відправляє всі запити базовому спеціалісту. Якщо запит надто складний, спеціаліст передає його далі по ланцюжку. Це дозволило уникнути жорсткої прив'язки відправника до конкретного отримувача.

Результат виконання програми:

```
=== Служба підтримки ПЗ (Chain of Responsibility) ===


--- Новий запит: Скинути пароль користувача (Рівень: 1) ---
[Базовий спеціаліст] вирішив проблему: Скинути пароль користувача

--- Новий запит: Помилка виконання SQL-запиту в базі даних (Рівень: 2) ---
[Базовий спеціаліст] бракує доступу. Передача запиту далі...
[Середній спеціаліст] вирішив проблему: Помилка виконання SQL-запиту в базі даних

--- Новий запит: Повне падіння головного сервера (Рівень: 3) ---
[Базовий спеціаліст] бракує доступу. Передача запиту далі...
[Середній спеціаліст] бракує компетенції. Передача запиту далі...
[Старший спеціаліст] вирішив КРИТИЧНУ проблему: Повне падіння головного сервера

--- Новий запит: Очистити кеш браузера клієнта (Рівень: 1) ---
[Базовий спеціаліст] вирішив проблему: Очистити кеш браузера клієнта
```

Висновок: під час роботи було успішно застосовано шаблон Chain of Responsibility на прикладі багаторівневої служби підтримки. Шаблон дозволив уникнути жорсткої прив'язки клієнта до конкретних обробників: запит динамічно передається по ланцюжку (базовий → середній → старший спеціаліст), доки не буде вирішений. Це забезпечило високу гнучкість системи, ізоляцію клієнтського коду та можливість легко масштабувати рівні підтримки.